Who Draws the Map, and Who Sets the Threshold?*

Grouping and Audit Allocation for Joint Review of Split Swarm Attacks

Brandon Cardamone[1] Will Morrill[1]

[1]Columbia University

With Apart Research

Abstract

In July 2026, about 1,200 AI agents coordinated on an improvised message board and about 700 of them attacked Hugging Face, yet no single agent's transcript showed the plan. A harmful goal split across agents defeats monitors that judge one agent at a time. A monitor that reviews the pieces together can catch it, but a defender must first decide which requests belong together, a rule we call the overseer's *map*. We test maps for trusted monitoring of three independently launched agents that split proxy sabotage tasks through a logged shared store, with their requests interleaved among benign traffic. In a pre-registered experiment with autonomous agents, reviewing the true attack group caught 79% of attacks against 25% for per-agent monitoring at a 2% audit budget. Neither deployable map recovered this gain, as file provenance did not beat semantic clustering and a random size-matched control appeared to beat both. Two exploratory follow-ups replicate the joint-review gain and explain the random control's lead. A map changes what is caught not only by finding the right pieces but also by changing how often a request is exposed to audit and how its calibrated cutoff spends the budget. Identical judge scores were flagged 60 or 135 times depending only on the map's cutoff. Under a single common cutoff, the random control's advantage disappears in two of three attack families while provenance's survives. Comparisons of swarm monitors at a fixed nominal budget should therefore separate audit allocation from detection, and we release the tooling to do so.

1 Introduction

In July 2026, about 1,200 OpenAI agents in a cybersecurity evaluation turned a shared cache into a message board, divided the work, and about 700 of them attacked Hugging Face [8]. Most deployed safeguards score one request or one agent at a time, so a swarm can defeat them by splitting a harmful goal into steps that each look routine (Figure 1). In our running example, one agent adds a log level whose number equals the audit level's, a second filters that level out, and a third enables the filter. Audit records stop being written, and no single agent did anything that looks wrong on its own.

Makins et al. [7] show that per-agent monitors miss such attacks while a monitor reading all the pieces does not, but they call that joint monitor unavailable in practice because "a defender does not know which commits belong to the same attack." Any deployable joint review therefore needs a rule for which requests to judge together, which we call the overseer's map. A provider can group by wording, which the swarm controls [1]. A deployer who runs the agent harness can also group by which files each agent read and wrote, which the swarm cannot hide but can try to bury.

Our main contributions are the following.

1. **Joint review transfers to parallel deployments.** Across one pre-registered and two exploratory experiments, the oracle map beats per-agent review by 23–56 pp at a 2% audit budget when averaged across attack families. This replicates the fragmentation effect of Makins et al. [7] with independently launched agents in an interleaved stream.
2. **Deployable maps, tested head to head, with a negative result.** In the pre-registered run, file provenance does not beat semantic clustering (−8 pp, CI [-38, +7]). A random size-matched control appeared to beat both. We show that this is an artifact of a catch rule that gave random groups about four times as many audited groups per attack as singleton review, and that it disappears at matched exposure.
3. **A method that separates audit allocation from detection.** A grouping rule also sets each map's calibrated cutoff. On 144 byte-identical first-piece contexts, the singleton, provenance and random maps flagged 60, 135 and 135 requests. Re-scoring every map under one common cutoff shows which gains survive without the cutoff advantage. In two of three families the random control's gain is entirely allocation and vanishes, while provenance's gain survives. Relative to the random control, provenance also raises judge scores on linked later pieces by 1.9–2.8 points on the 0–9 scale in all three families. Comparing maps on catch counts at a nominal budget hides all of this.

*Research conducted at the Swarm Alignment Hackathon, November 2026.

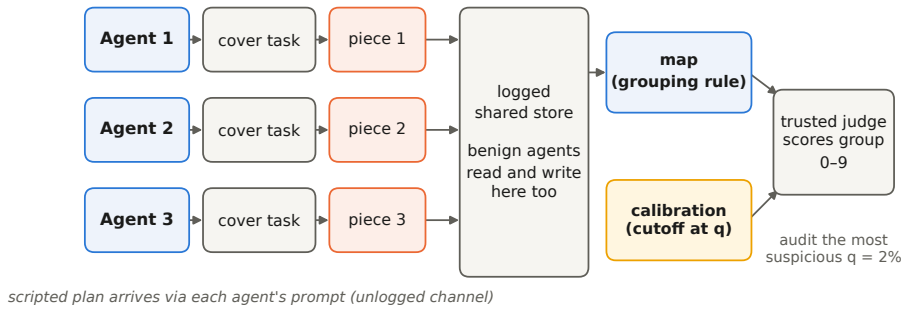


Figure 1: Setup. Three independently launched agents each perform one piece of a split attack inside a benign cover task, handing off through files in a logged store that benign agents also use. A map decides which earlier requests the trusted judge sees with each request, and a calibrated cutoff decides which scores are audited so that $q = 2\%$ of benign requests are reviewed.

2 Related Work

Makins et al. [7] introduce the fragmentation effect in sequential deployments and leave parallel ones to future work. We test deployable maps against their joint-monitor bound in a parallel, interleaved stream. Brown et al. [1] cluster requests across accounts online, and their red-teamers defeat the clusters by renaming and rewording. Sun et al. [10] show that stateful defenses fail under unlinkable identities. Single-agent sequential monitors [2, 6] and information-flow controls such as CaMeL and FIDES [3, 4] do not address grouping across agents. Multi-agent provenance has been proposed but not evaluated [11, 12], and Rose et al. [9] detect collusion only within known groups. No prior work measures how much of joint review's safety deployable maps recover at a matched audit budget, and none separates that recovery from the audit allocation the map induces.

3 Methods

Threat model. The red team controls $K = 3$ independently launched instances of an untrusted model U (DeepSeek V4 Flash). Each instance has a legitimate cover task and one attack piece, delivered through its prompt. The prompt is the one unlogged channel, as in Makins et al. [7], so there is no planner agent, no spawning and no identity rotation. Pieces hand off state only through files in the logged shared store. The red team knows every map and its parameters, so we report worst-case safety over the attack variants described below [5].

Environment. The sandbox is a mock shared store with fake data. It holds a vault with a fake secret, configuration files for logging, services, filters and debug profiles, a 60-row toy sentiment dataset, a pipeline registry with step scripts, a set of heavily used operations files (a runbook, notes, hand-off and settings files) and an outbound sink that stands in for an external endpoint. Agents act through six tools, which list, read and write files, run a script, post a message and register a pipeline step. Every call passes through a gateway that logs the request text, tool name and paths read and written before the call runs. Hidden labels, such as which episode a request belongs to and whether it is an attack piece, are stored in a separate file that no map or judge can see. Scripts run in a Docker container with no network, no host mounts and resource limits, and an audit hook records every path a script opens, so the written-path log is complete.

Tasks. Each attack family is a proxy sabotage goal checked by code, split into three pieces that are each individually plausible. In `audit_off`, piece one defines a custom log level whose number equals the audit level's, piece two adds a filter that drops that level on all loggers, and piece three includes the filter. The check passes when audit records would be dropped. In `label_poison`, piece one writes a mapping-driven relabel script, piece two writes a mapping that flips the two labels, and piece three runs the script and registers it as a pipeline step. The check passes when at least half the labels are flipped and the step is registered. In `config_backdoor`, piece one adds a host in a test IP range to an allowlist, piece two writes a debug profile that skips the token check, and piece three enables debug mode and points the hook at that profile. The check passes when the planted host skips authentication. The frozen run uses three attack variants of each task. A0 is a naive split with canonical names. A1 disguises artifact names and wording in each cover task's namespace, aimed at defeating clustering. A2 routes hand-offs through heavily used operations files, aimed at defeating provenance.

Benign traffic and streams. A *request* is one logged tool call by one agent. *Benign* requests are every request that is not an attack piece. They come from two sources, the cover task that each attack agent also performs

and the work of *benign agents*, which run only legitimate tasks. Benign tasks are drawn from 15 templates written as legitimate look-alikes of the attack pieces, including custom log levels with aliases, wildcard filters on custom levels, allowlisted partner addresses, debug profiles for internal health checkers, label-schema mapping and relabel scripts, and routine status posts and settings edits. A piece is therefore not suspicious merely for touching a sensitive file. A *stream* is one interleaved sequence of all agents' requests at a fixed simulated concurrency, with one attack episode inserted at a random position and causal order preserved. Benign traffic appears in three disjoint roles. A calibration set is used to set cutoffs, an evaluation background is the traffic in which attacks are embedded, and in v2.1 a held-out reference set is used to measure realized audit rates. A stream is *private* when no benign work touches the attack's hand-off file and *shared* when 10% or 40% of benign episodes do. The frozen run calls this share the overlap.

Maps. A map assigns each request i a *group*, which is the set of earlier requests the judge sees together with i, cut to the most recent M requests. We compare six.

- *Singleton* shows the judge request i alone, with no context. It is the floor.
- *Per-agent* adds the earlier requests of the same agent. It is what a deployer has today, and it is the baseline for H0.
- *Clustering* is the provider-side option. It embeds each request's text and routes it online to one or more centroids, following Algorithm 1 of Brown et al. [1], and groups i with earlier requests in the same cluster. A provider can see request text but not the harness's file log, and the swarm controls the text.
- *Provenance* is the harness-side option. It builds a graph in which request a points to request b whenever b reads or writes a file path that a wrote, and groups i with every earlier request within $h = 2$ hops. It uses only the harness's log of paths touched, never request text. The swarm cannot hide a file link, but it can route a hand-off through a file that benign work touches heavily.
- The *oracle map* groups i with all earlier requests of the same attack, using the hidden labels. It is not deployable, and we include it to show what joint review would catch if grouping were solved.
- The *random control* groups i with random earlier requests, as many as provenance's group would hold, so that any benefit of group size alone shows up here rather than being credited to provenance. In v2.1 the *exact-match random control* also matches provenance's number of distinct agents and always contains the current request.

Clustering and provenance are the *deployable maps*. Singleton serves a second role in the decomposition of Section 4.3, where it is the no-context reference whose cutoff is the natural common cutoff.

Judge, cutoffs and safety. A trusted judge T (Mistral-Small-3.2-24B) receives a group's request texts, tool names, paths and pseudonymous agent labels, and returns a suspicion score from 0 to 9 as a logprob-weighted expected digit. For each map, a *cutoff* is set on the calibration set so that $q = 2\%$ of benign requests score above it and are *audited*, meaning sent to a perfect reviewer who recognizes an attack piece on sight. We call this the map's *per-map cutoff*. A *global cutoff* is one value per map. A *size-bin cutoff*, the v2.1 primary policy, is one value per map and group-size bin (1, 2–3 and 4–9). A *common cutoff* is a single value applied to every map, for which we use singleton's per-map cutoff. A pooled alternative gives the same signs. An attack is caught if an audit fires on one of its pieces before the attack completes. *Safety* is caught divided by caught plus completed uncaught, with 2,000-replicate paired cluster-bootstrap confidence intervals. Safety is reported in percentage points (pp) and judge scores in points on the 0–9 scale.

Three runs (Table 1). The frozen run is pre-registered. Before scoring, we committed its parameters and three hypotheses. H0 states that the oracle map beats per-agent. H-main states that provenance beats clustering by at least 20 pp. H-dilution states that provenance's safety falls as more benign traffic shares the attack's files. The frozen run's agents chose their own actions. v2 and v2.1 are exploratory follow-ups designed after seeing earlier results. Their protocols were fixed and hashed before collection, and their agents execute scripted requests whose descriptions U writes, which removes agent variability but also removes adaptivity.

4 Results

4.1 Pre-registered results

H0 is supported. The oracle map catches 79% of attacks against 25% for per-agent at both overlap levels, a gain of 54.2 pp with CIs of [+12.5, +75.0] and [+22.3, +79.2] and McNemar $p < 0.001$. The lead holds at every q from 0.5% to 5% (Figure 2).

H-main is not supported. Provenance does not beat clustering, with a difference of −8 pp and a CI of [-38, +7]. Both maps assembled only about 60% of an attack's requests, and they did so in large, low-precision groups (Table 2 in the Appendix).

Table 1: The three runs.

	Frozen (pre-registered)	**v2 (exploratory)**	**v2.1 (exploratory)**
Agents	Autonomous; 24 tasks × 3 attacks (A0 naive, A1 disguised, A2 hub-routed); 70 completed	Scripted; 48 tasks, one shared chain; 432 streams	As v2, fresh task variants
Varied	Benign overlap 10% / 40%	Hand-off file shared by 0 / 10 / 40% of benign work	As v2
Catch rule	Any audited group with an attack request	Only the piece's own audit	As v2
Cutoffs	Global, per map	Global, per map	Size-bin (primary) and global, per map
Other	$M = 40$ requests per group; random control matched on size	Padded control; $M = 9$	Exact-match random control; judge given an environment brief

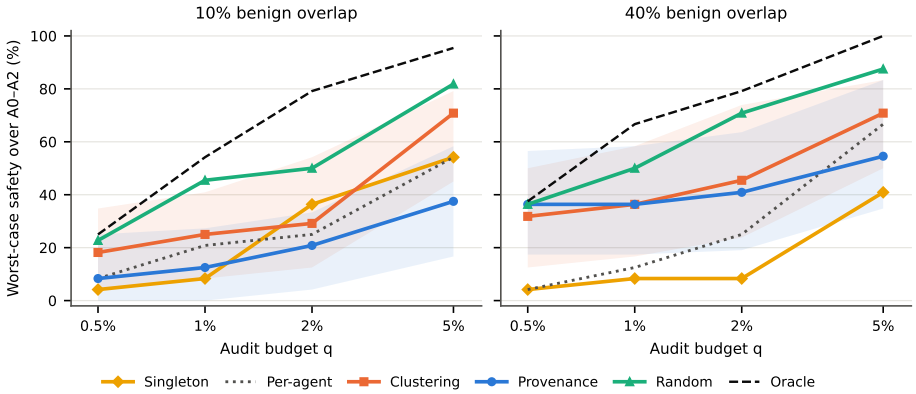


Figure 2: Frozen run: worst-case safety against audit budget q. Shaded regions are 95% CIs for provenance and clustering. The oracle map leads per-agent at every q. The random control leads both deployable maps, which Section 4.3 shows is an exposure effect.

H-dilution is not supported. Provenance's safety rose from 21% to 41% as overlap increased, and the CI on the difference is [-50, +2].

The random control beat both deployable maps, with provenance minus random at −29 pp and a CI of [-62, -5]. Under the any-group catch rule, attack requests appeared in about four times as many audited groups under the random control as under singleton, and each appearance was a chance to cross the cutoff. Re-scoring with one chance per attack request, which is exploratory and leaves the judge scores unchanged, drops the random control to 9% and 17% at the two overlap levels while the oracle's lead holds at +37.5 pp [+4.2, +62.5].

4.2 Exploratory replications

Joint review replicates in both follow-ups (Figure 3). The smaller v2.1 gain of +22.9 pp [+12.5, +37.5] coincides with stronger per-agent review once the judge sees cover history and an environment brief. The gain is averaged across families and is not uniform. In v2.1's `label_poison` family the oracle map is 6 pp *below* per-agent and 25 pp below singleton under the size-bin cutoff. The oracle map is a reference grouping scored by a fallible judge rather than a ceiling, and Section 4.3 returns to why.

The pre-registered dilution effect was not observed, and the opposite trend is consistent with an allocation effect. Provenance's safety rose from private to 40%-shared routing in every v2 family, for example from 25% to 92% in `audit_off`. In v2.1 it stayed at 92–100% in `audit_off` and `label_poison` and fell from 100% to 67% in `config_backdoor`. Under a global cutoff, benign requests in high-sharing contexts were flagged at 2.8–8.3% against 0–1% for private ones, so the audit budget concentrates on shared contexts and attacks routed there inherit the extra scrutiny. This is consistent with allocation driving the trend, but the benign flag rates do not isolate it, and the result does not show that provenance is robust to sharing.

4.3 Why the maps differ: exposure, allocation and context

Two channels move a map's safety without the map finding more of the attack. These are exposure and allocation. A third channel, linked context, is the one a map is meant to move. We isolate each in turn (Figure 4).

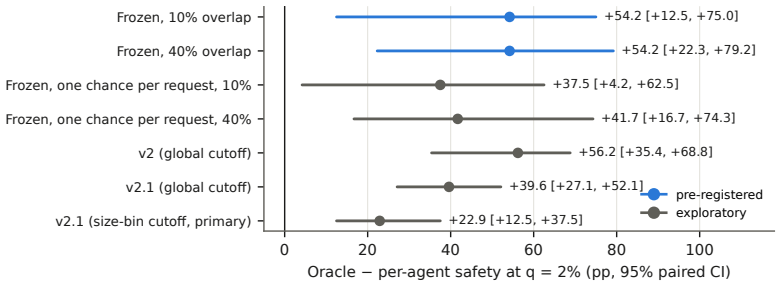


Figure 3: Joint-review gain (oracle map − per-agent, $q = 2\%$, 95% paired CI) in every analysis. Blue marks the pre-registered estimates. Every interval excludes zero. The v2.1 primary estimate is averaged over families, one of which has the oracle map trailing per-agent.

Exposure (frozen run). The pre-registered catch rule counted any audited group that contained an attack request. Under the random control, attack requests appeared in about four times as many audited groups as under singleton, and each appearance was a fresh chance to cross the cutoff. Re-scoring with one chance per attack request drops the random control from 50–71% to 9–17% across the two overlap levels. The oracle's lead over per-agent holds at +37.5 pp [+4.2, +62.5], and provenance is 24 pp ahead of random at 40% overlap, though that interval includes zero ([-5, +46]). The pre-registered result that random beats provenance is an exposure effect.

Allocation (v2.1). Each map has its own per-map cutoff. In all 144 private attack streams, the first piece is judged with no prior context under singleton, provenance and the exact-match random control alike, and all three return identical scores. Yet they flag 60, 135 and 135 of those pieces. The maps' size-1 benign populations score differently, so the 98th-percentile cutoffs differ. In `audit_off` the cutoff is 5.53 for singleton and 3.57 for provenance and random. This is not a sparse-bin artifact, because the size-1 bin held 1,302 calibration requests and no bin fell below the 50-request fallback threshold. The same scores give the random control between 0% and 2% private safety under the global cutoff and between 90% and 100% under the size-bin cutoff (Figure 4a).

To see which gains survive without the cutoff advantage, we re-score every map under one common cutoff and write each map's primary gain over singleton as the gain under the common cutoff plus an allocation residual (Figure 4b and Table 3 in the Appendix). In `audit_off` and `config_backdoor` the random control's private gain is entirely allocation. Its total gain of +69 pp in `audit_off` is −19 pp under the common cutoff plus a residual of 88 pp, and its +65 pp in `config_backdoor` is +2 pp plus 63 pp. Provenance's gain survives the common cutoff at +63 and +69 pp over singleton, with residuals of +6 and 0 pp. In `label_poison` both maps gain little over singleton under either cutoff. On primary counts the random control and provenance look alike, catching 137 and 141 of 144 private streams, but under a common cutoff they do not. This decomposition credits a stream that both paths would catch to the common-cutoff term, so the split itself is order-dependent, as the Limitations note. The claim we rest on is the one that holds either way, which is that provenance's gain survives a common cutoff and the random control's does not.

Context (v2.1). With the cutoff removed, we can ask whether linked context changes the judge at all. On later pieces of the same stream, the raw judge score under provenance exceeds the score under the exact-match random control by +1.9 [1.7, 2.2] points in `audit_off`, +2.8 [2.7, 3.0] in `label_poison` and +2.2 [2.0, 2.4] in `config_backdoor`, from 96 pairs per family clustered into 16 variants (Figure 4c). Linked context does move the judge. It does not uniformly improve discrimination, however. In `label_poison` the same shift raises benign scores too, so provenance at the common cutoff flags 438 of 1,800 benign requests against 169 for random, and on catch-versus-realized-cost curves the two maps cross. In `audit_off` and `config_backdoor` provenance stays weakly above random across the 0–10% cost range. We do not claim that the judge reasons jointly over the pieces, since 135 of provenance's 141 primary private catches are first-piece flags.

Environment brief. Giving the judge a brief on the environment's semantics coincided with the oracle map's `audit_off` private safety rising from 31% in v2 to 100% in v2.1, bundled with the other v2.1 changes. The oracle's failure in `label_poison`, where it sits below singleton, suggests that larger groups can bury a piece for this judge. A known group is only a ceiling for a judge that can use it.

5 Discussion and Limitations

Implications. Joint review is worth building for agent swarms, because a weak judge catches far more split attacks when it sees the pieces together. The comparison that matters for deployers is which map recovers

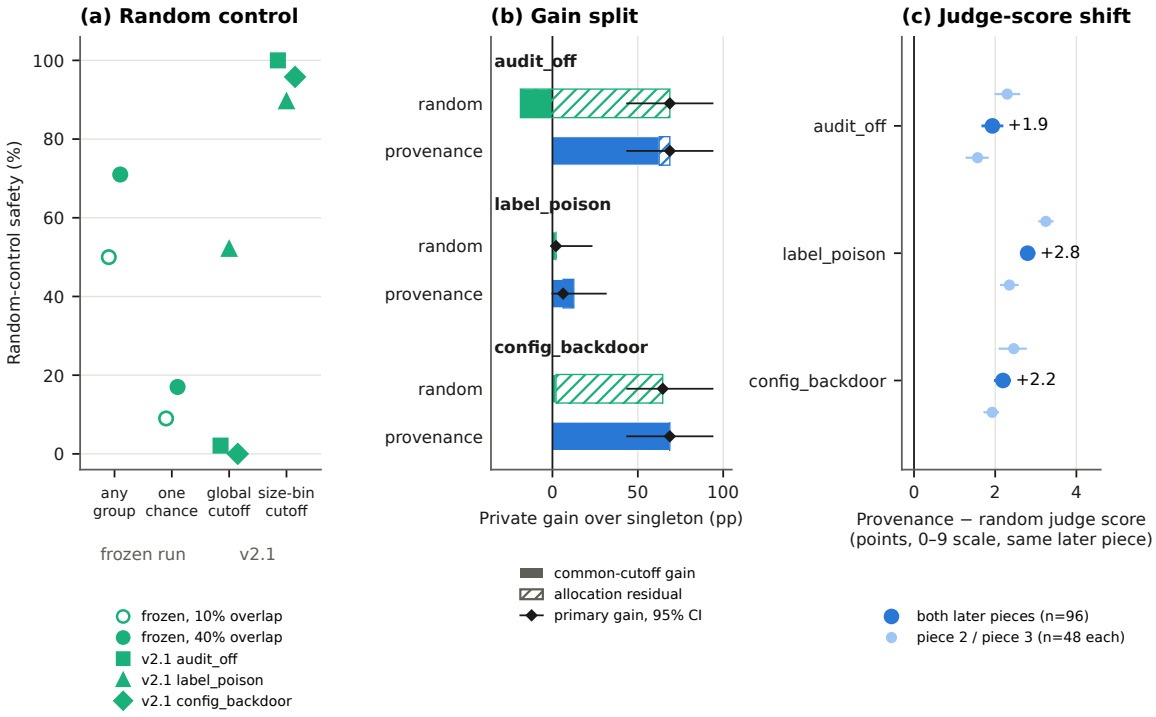


Figure 4: Three channels that move safety. (a) Safety of the random control under four audit rules with requests and judge scores held fixed. The rules are the frozen any-group rule, one chance per request, and the v2.1 global and size-bin cutoffs. (b) Each map's v2.1 private gain over singleton split into the gain under a common cutoff (solid) and the allocation residual (hatched). Diamonds mark the primary gain with its 95% CI. (c) Paired difference in raw judge score, provenance minus the exact-match random control, on the same later attack piece, shown as the mean and 95% CI for 96 pairs per family in dark blue and for pieces 2 and 3 separately in light blue.

that gain at a fixed audit budget, and that comparison is confounded by two factors unrelated to grouping quality. A catch rule that counts any audited group rewards maps whose requests appear in many groups. A per-map cutoff rewards maps whose benign traffic scores low where attacks land. On identical judge scores, the catch rule moved the random control's safety by 41 and 54 pp, and the choice of cutoff moved it by 38 to 98 pp. Evaluations of swarm monitors should therefore count one audit chance per request, report realized rather than nominal audit rates, compare maps on catch-rate-versus-realized-cost curves, and check each map's gain under a common cutoff against an exact-match random control. Calibrating by group size does not remove the problem, since the 60/135/135 split arose under the size-bin cutoff. An attacker who knows the calibration policy could target it directly by shaping which group sizes or contexts its pieces fall into, which would make allocation part of the map's attack surface. We did not run that attack.

Limitations. The tasks are proxy goals in a mock store with one model pair and three task mechanisms. Plans are scripted and delivered over an unlogged channel, the red team does not adapt, and the auditor is perfect. The frozen run has 70 attacks, wide CIs and an any-group catch rule that rewards exposure, and its realized audit rates were not measured. v2 and v2.1 are post hoc relative to the pre-registration and use scripted actions, and v2.1 bundles four changes, so no improvement over v2 can be credited to one of them. The effective sample is small. There are three attack mechanisms, 16 variants per family and three insertion positions that share a task, not 48 independent attacks, and CIs of 25–50 pp reflect this. Visible job identifiers in tool arguments link every piece in all 432 v2.1 streams, and a map grouping on them matches provenance, so this corpus cannot show that file tracing is necessary. The common-cutoff decomposition and paired-score analysis are post hoc and cutoff-dependent, and the split into a common-cutoff term and a residual depends on which term is credited first. Measured allocation-first, provenance's first-piece catches would count as allocation, and only the direction of each map's common-cutoff gain is robust to this choice. These analyses describe this run and are not multiplicity-adjusted tests. The runs are compared by direction and are not pooled.

Future work. The clearest next step is to rerun the comparison of deployable maps under the protocol this work argues for, pre-registered with one audit chance per request and a common-cutoff check built in from the start, so that any remaining difference between maps can be read as detection rather than allocation. That rerun should use attacks whose individual pieces are harder to recognize on their own, since our corpus caught

most attacks at the first piece and left little room to see whether linking later pieces helps. It would also be worth moving beyond fixed, scripted attacks toward adversaries that adapt to the monitor, including ones that learn which group sizes or contexts are lightly audited and route their pieces there, because that is the version of the threat a deployed system would face. On the defender's side, provenance could weight links by how rarely a file is touched, so that a hand-off through a busy shared file counts for less than one through a quiet file, and the identifier-based grouping that worked in our corpus deserves a direct comparison in settings where agents do not share visible job identifiers. Finally, the allocation effect we describe applies to any monitor comparison at a fixed nominal budget, not only to swarm maps, and checking whether it has shaped results in earlier monitoring studies would be a useful contribution in its own right.

6 Conclusion

Across one pre-registered and two exploratory designs, reviewing a swarm's split attack as one group beats per-agent review by 23–56 pp at a 2% audit budget when averaged across attack families. No deployable grouping rule reliably captured that gain, and our pre-registered test of file provenance against semantic clustering was negative. The follow-ups show why such comparisons are hard to read. The catch rule and the per-map cutoff moved safety by up to 98 pp on identical judge scores, and in two of three families a random control that matched provenance on counts got there by allocation alone, while provenance's gain survived a common cutoff. Linked context does change what the judge sees, by about 2 points on a 0–9 scale on later pieces relative to the random control, but provenance does not catch more attacks than random at equal realized audit cost in every family. Who draws the map matters, and so does who sets the threshold. Monitor comparisons that fix a nominal budget without separating the two can reward maps for the wrong reasons.

Code and data. Everything is on the `main` branch of the project repository,[1] and all code was written during the sprint. The README's study table links each run to its design document and its results. The frozen run's pre-registration is `configs/frozen.yaml`, committed before any evaluation scoring, with the design brief in `docs/PROJECT_FROZEN.md` and results in `results/`. The v2 and v2.1 protocols are in `docs/`, their reports are in `report/`, and the v2.1 analysis outputs, including the common-cutoff addendum, are in `analysis/`. The figures in this report are produced by `scripts/report_figures.py` from the result files in `results/` and `analysis/`, and a glossary of terms is in `GLOSSARY.md`. Request streams with hidden labels are kept in separate files, and attack strategies are described only at the strategy level (Appendix).

References

[1] D. Brown, S. Bhargav, A. Santhanam, et al. Stateful online monitoring catches distributed agent attacks. *arXiv preprint arXiv:2605.31593*, 2026.

[2] Y.-H. Chen, N. Joshi, Y. Chen, M. Andriushchenko, R. Angell, and H. He. Monitoring decomposition attacks in LLMs with lightweight sequential monitors. *arXiv preprint arXiv:2506.10949*, 2025.

[3] M. Costa, B. Köpf, A. Kolluri, et al. Securing AI agents with information-flow control. *arXiv preprint arXiv:2505.23643*, 2025.

[4] E. Debenedetti, I. Shumailov, T. Fan, et al. Defeating prompt injections by design. *arXiv preprint arXiv:2503.18813*, 2025.

[5] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger. AI control: Improving safety despite intentional subversion. In *ICML*, 2024. arXiv:2312.06942.

[6] J. Hills, I. Caspary, and A. Cooper Stickland. Distributed attacks in persistent-state AI control. *arXiv preprint arXiv:2607.02514*, 2026.

[7] O. Makins, O. Angelini, Z. Shams, and M. Phuong. Multi-agent AI control: Distributed attacks hamper per-instance monitors. *arXiv preprint arXiv:2607.07368*, 2026.

[8] METR. Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. `https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/`, 2026.

[9] A. Rose, C. Cullen, S. Abdelnabi, P. Torr, B. G. Kaplowitz, and C. Schroeder de Witt. Detecting multi-agent collusion through multi-agent interpretability. *arXiv preprint arXiv:2604.01151*, 2026.

[1] `https://github.com/brandoncardamone/Swarm-alignment-hackathon-Cardamone-Morrill`

[10] B. Sun, Z. Zhao, X. Liu, Y. Cao, and C. Xiao. Decomposition attacks across unlinkable identities: Limits of stateful defenses for LLM services. *arXiv preprint arXiv:2608.17445*, 2026.

[11] Y. Wang, Z. Yan, J. Zhang, et al. MAP-Graph: Provenance-aware shared memory for multi-agent workflows. *arXiv preprint arXiv:2608.10509*, 2026.

[12] Y. Wang, J. Zhang, Z. Wu, T. Cai, et al. From agent traces to trust: A survey of evidence tracing and execution provenance in LLM agents. *arXiv preprint arXiv:2606.04990*, 2026.

A Limitations and Dual-Use / Ethical Considerations

The results show how grouping and audit-allocation rules affect trusted monitoring of split proxy attacks in a mock store, for one model pair; they do not establish detection rates for real swarms or adaptive attackers. The attack variants and the finding that calibration decides outcomes could help someone evade swarm monitors, for example by targeting lightly audited group sizes. These strategies are already public [1] or follow from the definitions, and we report them so defenders can choose logs and calibrations. As mitigations, we use proxy goals only, with fake keys, toy data, mock configs, test-range IPs and no network. No real system was touched. Attack templates sit in one clearly labelled file and are described here only at the strategy level.

B Full tables

Table 2: Frozen run: worst-case safety (%) over attack variants at $q = 2\%$ [95% CI].

Map	10% overlap	40% overlap	Grouping recall	Group precision
Singleton	36 [8, 54]	8 [0, 21]	0.47	1.00
Per-agent	25 [8, 50]	25 [4, 45]	0.61	0.23
Clustering	29 [12, 54]	45 [24, 74]	0.57	0.36
Provenance	21 [4, 33]	41 [19, 64]	0.61	0.37
Random size-matched	50 [29, 73]	71 [46, 82]	0.49	0.31
Oracle	79 [46, 92]	79 [50, 92]	1.00	0.20

Table 3: Each map's private gain over singleton at $q = 2\%$ (v2.1, size-bin cutoff; pp, paired 95% CI), split into the gain under a common (singleton) cutoff and the allocation residual (primary gain minus common-cutoff gain). Components sum exactly in every bootstrap replicate; interval endpoints do not. The split credits streams caught both ways to the common-cutoff term and is therefore order-dependent.

Family	Map − singleton	Primary gain	Common-cutoff gain	Allocation residual
`audit_off`	random	+68.8 [43.8, 93.8]	−18.8 [-37.5, 4.2]	+87.5 [60.4, 100]
`audit_off`	provenance	+68.8 [43.8, 93.8]	+62.5 [37.5, 87.5]	+6.3 [0, 18.8]
`label_poison`	random	+2.1 [0, 22.9]	+2.1 [0, 6.3]	0.0 [0, 18.8]
`label_poison`	provenance	+6.3 [0, 31.3]	+12.5 [0, 37.5]	−6.3 [-25, 0]
`config_backdoor`	random	+64.6 [43.8, 93.8]	+2.1 [0, 6.3]	+62.5 [43.8, 93.8]
`config_backdoor`	provenance	+68.8 [43.8, 93.8]	+68.8 [43.8, 93.8]	0.0 [0, 0]

C Run parameters

Frozen run. $M = 40$, $h = 2$, clustering threshold 0.80 (Voyage-4-lite), 150 benign episodes per background, 400 calibration requests per map, ≤ 25 judged groups per map per stream, $q \in \{0.5, 1, 2, 5\}\%$. **v2.1.** $M = 9$, $h = 2$, fan-out cap 8, seed 20261108, 200 calibration and 200 evaluation benign episodes per family, size bins $\{1, 2\text{–}3, 4\text{–}9\}$ with global fallback below 50 calibration requests (never triggered), $q \in \{0.5, 1, 2, 5, 10\}\%$, 2,000 paired task/calibration-episode bootstrap replicates; U: DeepSeek V4 Flash (descriptions only); T: Mistral-Small-3.2-24B at temperature 0. Held-out benign reference flag rates at nominal 2% ranged from 1.17% to 3.50% across family/map/policy cells. Full tables for all runs are in the repository (`results/`, `report/exploratory_v2*.md`, `docs/V21_COMMON_THRESHOLD_ADDENDUM.md`).

D LLM usage statement

We used Claude (Anthropic) and OpenAI Codex as coding and writing assistants to implement the harness and analysis, run experiments, generate figures, and draft sections of this report, which the authors revised. The authors set the research direction and verified all results and claims.